常见加速方法： 二分，倍增，根号分治，矩阵快速幂， FFT/NTT/FWT， sosdp，求封闭表达式O（1）计算

基本

- 二分
- 快速幂
- 线性求逆元

数据结构

- 树状数组
- 线段树
- 没有lazy_tag的线段树
- 珂朵莉树
- Segment_set
- 可持久化线段树
- LCA
- LCA更新
- 判断边在不在树的链上
- 黑科技LCA
  $O(nlog(n))$预处理，在线$O(1)$ 查询， 类dfs序，ST实现的RMQ
- 笛卡尔树
- 树剖
- dfs序
- 扩展dfs序
- 树上的RMQ与LCA
- 欧拉序（常用于树上莫队，树链求异或和）

图论

- 并查集（路径压缩）
- 并查集（按秩合并）
- 带权并查集
- 最小生成树
  - Kruskal
  - kruskal 重构树
- Dijkstra
- floyd
- Bellman-Ford
- Kuhn匈牙利算法

计算几何

- 基本
- 极角排序
- 旋转卡壳
- 凸包 (Convex Hull)
- 凸包
- 半平面交
  - 凸多边形

字符串

- 字符串哈希
- 字典树
- KMP
- Manacher
- AC自动机

dp

- SOSdp（高维前缀和）
  - 子集和dp
  - 超集和dp
- 二进制下枚举子集
- 树上背包 $(O(n^2))$
- 01背包
- 完全背包
- 多重背包
  - Naive Algorithm

        单调队列优化O(nm)

数学

    exgcd
    线性筛
    积性函数
    莫比乌斯反演
        1. 莫比乌斯函数的定义
        2. 莫比乌斯变换与反演公式
            (1) 狄利克雷卷积形式
            (2) 正变换
            (3) 逆变换
        3. 常见应用
            (0) ε的展开
            (1) 因子和反演
            (2) 卷积单位元表示
            (3) 欧拉函数恒等式
            (4) 计数互素对
        常见莫比乌斯反演恒等式
            1. 基础恒等式
            2. 反演恒等式
            3. 常见和式公式
            4. 与积性函数关系
    矩阵乘法与矩阵快速幂
    高斯消元
    线性基 （寻找第k大有些问题)
    Luca定理
    斯特林数
    多项式 写在前面
    DFT(离散傅里叶变换，discrete Fourier transform)
    IDFT
    简洁的fft
    简洁的ntt
    常见模数
    FWT
    生成函数
        展开与系数（给定 N）

博弈轮

    SG 函数模板 (Sprague–Grundy Theorem)
        1. 定义
        2. mex 函数实现
    Bash Game

杂项

    三分法 (Ternary Search)
        实数域三分法
        整数域三分法
    曼哈顿距离 ↔ 切比雪夫距离的坐标变换
    更高精度开根
    内存计算
    常见函数上下界
    常见公式
    斐波那契数列在算法竞赛中的常用性质
        基本性质
        模运算与周期性
        数学恒等式
        组合与数论应用
        竞赛常见考点

常见加速方法： 二分，倍增，根号分治，矩阵快速幂， FFT/NTT/FWT，sosdp，求封闭表达式O（1）计算

基本

二分

```
int getmax(int l , int r)  {
    int ans = -1;
    while(l <= r) {
        int mid = l + (r - l)/2;
        if(check(mid)) {
            l = mid + 1;
           ans = mid;
           }
        else r = mid - 1;
    }
    return ans;
}

int getmin(int l , int r ) {
    int ans = -1;
    while(l <= r) {
        int mid = l + (r - l)/ 2;
        if(check(mid)){
            r= mid - 1;
            ans = mid;
        }
        else l = mid + 1;
    }
    return ans;
}
```

快速幂

```
typedef long long ll;

int fastpow(int a, int b, int MOD) {
    ll res = 1;
    ll base = a % MOD;
    while (b) {
        if(b & 1) res = res * base % MOD;
        base = base * base % MOD;
        b >>= 1;
    }
    return (res + MOD) % MOD ;
}

int inv(int x , int MOD){   //MOD是质数
    return fastpow(x, MOD - 2, MOD) % MOD ;
}
```

线性求逆元

```
#include <bits/stdc++.h>
#define int long long
```

```cpp
using namespace std;

const int N = 3e6 + 10;

int n, p, inv[N];
signed main() {
    cin >> n >> p;
    inv[1] = 1;
    for (int i = 2; i <= n; i++)
     inv[i] = (p - (p / i) * inv[p % i] % p) % p;

    for (int i = 1; i <= n; i++)
     cout << inv[i] << "\n";
    return 0;
}
```

数据结构

树状数组

```
template<typename T> struct BIT{
    int len;
    vector<T>tr;
    BIT(int n=0){//给定n初始化
        len=n;
        tr.resize(len+1,0);
    }
    int lowbit(int x){
      return x&-x;
    }
    void init(int n,vector<T>a){//给定n和序列初始化
        len=n;
        tr.resize(n+1,0);
        for(int i=1;i<=n;i++) update(i,a[i]);
    }
    void update(int pos,T val){
     for(; pos<=len; pos+=lowbit(pos) ) tr[pos]+=val;
    }
    void update(int l,int r,T val){//对于差分序列,区间和为单点值,配合query(pos)使用
        update(l,val);
        update(r+1,-val);
    }
    T query(int pos){
        T sum=0;
        while(pos){
            sum+=tr[pos];
            pos-=lowbit(pos);
        }
        return sum;
    }
    T query(int l,int r){
        return query(r)-query(l-1);
    }
};
```

线段树

注意：如果是类似区间赋值的pushdown, 由于不存在单位元，需要lazy_tag和lazy值两个变量

```
class SegmentTree{
    struct node{
        int l, r;
        int val;
        int len;
        int lazy = 0;
    };

//lazy的定义：i的lazy第i个结点已经加过，要传给i << 1和 i << 1 | 1
public:
    SegmentTree(vector<int> _a){
        this-> n = _a.size() - 1;
```

```
    this-> a = _a;
    this-> T.resize(n << 2);
    build(1, 1, n);
}

void pushup(int i){
    T[i].val = T[i << 1].val + T[i << 1 | 1].val;
}

void build(int i, int l, int r){
    T[i].l = l; T[i].r = r;
    T[i].len = r - l + 1;
    if(l == r){
        T[i].val = a[l];
        return;
    }
    int mid = l + r >> 1;
    build(i << 1, l, mid);
    build(i << 1 | 1 , mid + 1 , r);
    pushup(i);
}

void pushTag(int i, int val){
    T[i].val += (T[i].len) * val;
    T[i].lazy += val;
}

//注意最后不要误加pushup,叶子时不能pushup
void pushdown(int i){
    if(T[i].lazy){ // 需要判断是否是单位元
        if(!(T[i].l == T[i].r)){
            pushTag(i << 1, T[i].lazy);
            pushTag(i << 1 | 1, T[i].lazy);
        }
        T[i].lazy = 0;
    }
}

void update(int i, int L, int R, int val){ // 注意最后pushup
    pushdown(i);
    if(T[i].r < L || T[i].l > R) return; //没有交集
    if(T[i].l >= L && T[i].r <= R){ //包含全部信息，加完后return
        pushTag(i, val);//相当于加
        return;
    }
    update(i << 1, L, R, val);
    update(i << 1| 1, L, R, val);
    pushup(i);
}

int query(int i, int L, int R){
    pushdown(i);
    if(T[i].r < L || R < T[i].l) return 0;  //没有交集
    if(L <= T[i].l && T[i].r <= R) return T[i].val; //包含全部信息，return
    return query(i << 1, L, R) + query(i << 1 | 1, L , R);
}

int n;
vector<int> a;
```

```
    vector<node> T;
};

void solve(){
    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for(int i = 1; i <= n; i++) cin >> a[i];
    SegmentTree segTree(a);
    while(q--){
        int op; cin >> op;
        if(op == 1){
            int l, r, val;
            cin >> l >> r >> val;
            segTree.update(1, l, r, val);
        }else{
            int l, r; cin >> l >> r;
            cout << segTree.query(1, l, r) << endl;
        }
    }
}
```

没有lazy_tag的线段树

```
#include <bits/stdc++.h>
using namespace std;

struct SegmentTree{
    struct node{
        int l, r;
        int val;
    };
    int n;
    vector<int> a;
    vector<int> mp; //找到数组下标对应的叶子区间编号
    vector<node> seg;
    // lazy的定义：i的lazy 表示第 i 个结点已经加过，要传给 i<<1 和 i<<1|1
    SegmentTree(vector<int> _a){
        n = (int)_a.size() - 1;
        a = _a;
        seg.resize(n << 2);
        mp.resize(n + 1);
        build(1, 1, n);
    }
    void pushup(int i){
        seg[i].val = seg[i << 1].val + seg[i << 1 | 1].val;
    }
    void build(int i, int l, int r){
        seg[i].l = l; seg[i].r = r;
        if(l == r){
            seg[i].val = a[l];
            mp[l] = i;
            return;
        }
        int mid = (l + r) >> 1;
        build(i << 1, l, mid);
        build(i << 1 | 1 , mid + 1 , r);
        pushup(i);
```

```
    }

    void update(int pos, int val){ // 注意最后pushup
        int ind = mp[pos];
        seg[ind].val += val;
        while(ind /= 2) pushup(ind);
    }

    int query(int i, int L, int R){
        if(seg[i].r < L || R < seg[i].l) return 0;         // 没有交集
        if(L <= seg[i].l && seg[i].r <= R) return seg[i].val; // 完全包含
        return query(i << 1, L, R) + query(i << 1 | 1, L , R);
    }
};
```

珂朵莉树

```
// seg[l] = {r, info}
// 表示一个线段 [l, r]，附加信息为 info
map<int, pair<int, int>> seg;

auto split(int p)
{
    auto it = seg.upper_bound(p);
    if (it == seg.begin()) return it;
    --it;
    auto l = it->first;
    auto [r, info] = it->second;
    if (l > p || r < p) return seg.end();
    if (l == p) return it;
    it->second.first = p - 1;
    seg[p] = {r, info + p - l};
    return seg.find(p);
}

// 区间操作
    auto itL = split(l);
    auto itR = split(r + 1);

    // 遍历 [l, r] 内的所有线段
    for (auto it = itL; it != itR; ++it) {
        int x = it->first;
        auto [y, info] = it->second;
        // 当前线段为 [x, y]，附加信息为 info
    }

    // 删除 [l, r] 内的所有线段
    seg.erase(itL, itR);

    // 插入新线段
    seg[l] = {r, info};

//
```

Segment_set

```cpp
/*
 every pair of pair<int,int> will not intersect
 if i is true then it will be in the set
 */
 struct SegmentSet {
  set<pair<int, int>> s;
  void write() {
    for (auto [x, y] : s) cout << x << "/" << y << " "; cout << "\n";
  }
  // make l...r to be true
  void insert(int l, int r) {
    int L = l, R = r;
    auto it = s.lower_bound(make_pair(L, (int)-2e9));
    while (it != s.end() && it->first <= R + 1) {
      R = max(it->second, R);
      it = s.erase(it);
    }
    if (it != s.begin()) {
      it--;
      if (it->second + 1 >= L) {
        L = min(L, it->first);
        R = max(R, it->second);
        s.erase(it);
      }
    }
    s.insert(make_pair(L, R));
  }
  // if l...r all true return false
  bool query_no_full(int l, int r) {
    auto it = s.lower_bound(make_pair(l, (int)-2e9));
    if (it != s.end()) {
      if (l == it->first && r <= it->second) return false;
    }
    if (it != s.begin()) {
      it--;
      if (it->second >= r) return false;
    }
    return true;
  }
  // make l...r to be false
  void del(int l, int r) {
    auto it = s.lower_bound(make_pair(l, (int)-2e9));
    while (it != s.end() && it->first <= r) {
      if (it->second <= r) {
        it = s.erase(it); continue;
      }
      int R = it->second;
      s.erase(it);
      s.insert(make_pair(r + 1, R));
      break;
    }
```

```
    it = s.lower_bound(make_pair(l, (int)-2e9));
    if (it != s.begin()) {
      it--;
      int L = it->first, R = it->second;
      if (R >= l) {
        s.erase(it);
        if (L <= l - 1) s.insert(make_pair(L, l - 1));
        if (R >= r + 1) s.insert(make_pair(r + 1, R));
      }
    }
  }

  // if l...r all false return false
  bool query_at_least_one(int l, int r) {
    auto it = s.lower_bound(make_pair(l, (int)-2e9));
    if (it != s.end()) {
          if (it->first <= r) return true;
    }
    if (it != s.begin()) {
        it--;
          if (it->second >= l) return true;
    }
    return false;
  }

 };
```

可持久化线段树

```
// 可持久化数据结构，动态开点，ls 和 rs 没有修改，只用修改新版本链上的 val
// node_idx为线段树用到的最大结点编号， root[i]为每个版本的线段树的根节点编号

class PersistentSegmentTree {
    struct node {
        int ls = 0, rs = 0; // 左儿子和右儿子的节点编号
        int val = 0;        // 叶子节点存储的值
    };
    int n;                  // 数组大小
```

```
    int node_idx = 0;       // 全局节点编号计数器
    std::vector<node> tree; // 节点池，存储所有节点

public:
    std::vector<int> root; // root[i] 存储版本 i 的根节点编号
    // 构造函数，接收初始数组并构建版本0
    PersistentSegmentTree(const std::vector<int>& _a) {
        this->n = _a.size() - 1;
        //  初始N个点 + M次操作，每次最多logN个新点
        this->tree.resize((n + 1e6 + 10) * 24);
        this->root.resize(1e6 + 10);
        // 构建初始版本 0
        build(this->root[0], 1, this->n, _a);
    }

    // 递归构建初始版本
    void build(int& nd, int l, int r, const std::vector<int>& a) {
        nd = ++this->node_idx;
        if (l == r) {
            this->tree[nd].val = a[l];
            this->tree[nd].sum = a[l];
            return;
        }
        int mid = l + (r - l) / 2;
        build(this->tree[nd].ls, l, mid, a);
        build(this->tree[nd].rs, mid + 1, r, a);
        push_up(nd);

    }
    // 公共接口：在某个版本上修改，并返回新版本的根节点编号
    // v_old: 基于哪个版本进行修改
    // p:     要修改的数组下标
    // c:     要修改成的值
    // 返回值: 新创建的根节点编号
    int update(int v_old_root, int p, int c) {
        int new_root = 0;
        upd(new_root, v_old_root, 1, this->n, p, c);
        return new_root;
    }

    void push_up(int nd){
        int ls = this->tree[nd].ls;
        int rs = this->tree[nd].rs;
        this->tree[nd].val = this-> tree[ls].val + this->tree[rs].val;
    }

    // 递归实现单点修改，并创建新版本
    void upd(int& nd, int old_node, int l, int r, int pos, int val) {
        nd = ++this->node_idx; // 创建新节点
        this->tree[nd] = this->tree[old_node]; // 复制旧节点信息

        if (l == r) {
            this->tree[nd].val = val;
            return;
        }


        int mid = l + (r - l) / 2;
        if (pos <= mid) {
```

```
            upd(this->tree[nd].ls, this->tree[old_node].ls, l, mid, pos, val);
        } else {
            upd(this->tree[nd].rs, this->tree[old_node].rs, mid + 1, r, pos, val);
        }

        push_up(nd);
    }

    // 公共接口：在指定版本上查询单个位置的值
    // v_root: 要查询的版本的根节点编号

    // p:      要查询的数组下标
    int query(int v_root, int p) {
        return query_recursive(v_root, 1, this->n, p);
    }


    // 递归实现查询
    int query_recursive(int nd, int l, int r, int pos) {
        if (l == r) {
            return this->tree[nd].val;
        }
        int mid = l + (r - l) / 2;
        if (pos <= mid) {
            return query_recursive(this->tree[nd].ls, l, mid, pos);
        } else {
            return query_recursive(this->tree[nd].rs, mid + 1, r, pos);
        }
    }
    //查询[L,R]区间和
    int query_sum(int nd, int l , int r, int L , int R){
        if(r < L || R < l) return 0;
        if(L < l &&  r <= R) return this->tree[nd].val;
        int ls = this->tree[nd].ls;
        int rs = this->tree[nd].rs;
        int mid = l + (r - l ) / 2;
        return query_sum(ls, l, mid,  L, R) + query_sum(rs ,mid + 1 , r,  L , R);
    }
};
```

LCA

```
const int MAXN = 100005; // 节点数量最大值
const int LOGN = 17;     // log2(MAXN)
vector<int> edge[MAXN];
int fa[MAXN][LOGN];//定于fa[x][0]为x的直接父节点

void dfs(int u, int p, int d) { // 第一个参数根节点，  第二个参数父节点，第三个参数深度，定义根节点深度为0
    depth[u] = d;
    fa[u][0] = p;
    for (int v : edge2[u]) {
        if (v != p) {
            dfs(v,u, d + 1);
            fa[v][0] = u;
        }
    }
```

```
}

void init(int n) {
    for (int k = 1; k < LOGN; ++k) {
        for (int i = 1; i <= n; ++i) {
            if (fa[i][k - 1] != -1) {
                fa[i][k] = fa[fa[i][k - 1]][k - 1];
            } else {
                fa[i][k] = -1;
            }
        }
    }
}
// 查询 LCA
int lca(int u, int v) {
    // 1. 保证 u 的深度不小于 v
    if (depth[u] < depth[v]) {
        swap(u, v);
    }
    // 2. 将 u 提升到和 v 相同的深度
    for (int k = LOGN - 1; k >= 0; --k) {
        if (depth[u] - (1 << k) >= depth[v]) {
            u = fa[u][k];
        }
    }
    // 3. 如果此时 u 和 v 相等，则 v (或 u) 就是 LCA
    if (u == v) {
        return u;
    }
    // 4. u 和 v 一起向上跳，直到它们的父节点相同
    for (int k = LOGN - 1; k >= 0; --k) {
        if (fa[u][k] != 0 && fa[u][k] != fa[v][k]) {
            u = fa[u][k];
            v = fa[v][k];
        }
    }
    // 5. 返回它们的直接父节点
    return fa[u][0];
}

int main() {
    int n, m;
    cin >> n >> m;
    int root = 1;
    for (int i = 0; i < n - 1; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    int rt = 1;
     dfs(rt,-1, 0); //定义根节点深度为0
    preprocess(n);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        cout << lca(u, v) << endl;
    }
    return 0;
```

```
}
```

LCA更新

```
int fa[N][25], dep[N];
//注意根节点的pre要取0
//fa[i][0]为i的直接父节点
void dfs(int u, int pre){
    dep[u] = dep[pre] + 1;
    fa[u][0] = pre;
    for(int i = 1; i < 25; i++) fa[u][i] = fa[fa[u][i - 1]][i - 1];
    for(auto v: edge[u]){
        if(v == pre) continue;
     dfs(v, u);
    }
}

int lca(int u, int v){
    if(dep[u] < dep[v]) swap(u, v); //钦定dep[u] >= dep[v]
    for(int i = 24; i >= 0; i--){
        if(dep[fa[u][i]] >= dep[v]) u = fa[u][i];
    }
    if(u == v) return u;
    for(int i = 24; i >= 0; i--){
         if(fa[u][i] != fa[v][i]) u = fa[u][i], v = fa[v][i];
    }
    u = fa[u][0], v = fa[v][0];
    return u;
}
```

判断边在不在树的链上

黑科技LCA

$O(nlog(n))$预处理，在线$O(1)$查询， 类dfs序，ST实现的RMQ

笛卡尔树

序列

```
p = [6, 3, 8, 1, 5, 2, 7, 4]
```

小根堆的笛卡尔树(括号内是结点value，@后面是位置)

```
            (1)@4
           /     \
       (3)@2      (2)@6
       /   \      /    \
   (6)@1  (8)@3 (5)@5  (4)@8
                        /
                    (7)@7
```

```
vector<int> L(n+1, 0), R(n+1, 0), fa(n+1, 0), stk;
for (int i = 1; i <= n; ++i) {
    int last = 0;
    while (!stk.empty() && a[stk.back()] > a[i]) { // 最小堆；排列可用 '>'
        last = stk.back();
        stk.pop_back();
    }
    if (last) {
        L[i] = last;
        fa[last] = i;
    }
    if (!stk.empty()) {  //当前元素为最大值
        R[stk.back()] = i;
        fa[i] = stk.back();
    }
    stk.push_back(i);
}
// root:
int root = 0;
for (int i = 1; i <= n; ++i) if (fa[i] == 0) { root = i; break; }

//L[i], R[i], fa[i]指的是原序列的下标
```

树剖

预处理

```
vector<int> dfn(n + 1), sz(n + 1), fa(n + 1), son(n + 1) , topf(n + 1), dep(n + 1);
dep[1] = 1;
function<void(int,int)> dfs = [&](int x, int f){
    sz[x] = 1;
    fa[x] = f;
    int maxi = -1;
    for(auto X: edge[x]){
        if(X == f) continue;
        dep[X] = dep[x] + 1;
        dfs(X, x);
        sz[x] += sz[X];
        if(sz[X] > maxi){
            maxi = sz[X];
            son[x] = X;
        }
    }
};
dfs(1, 0);
int id = 0;
vector<int> num(n + 1);
num[1] = 1;
function<void(int, int)> dfs1 = [&](int x, int top){
    dfn[x] = ++id;
    topf[x] = top;
    if(son[x] == 0) return;
    dfs1(son[x], top);
    for(auto X: edge[x]){
        if(X == son[x] || X == fa[x]) continue;
        dfs1(X, X);
    }
```

```
};
dfs1(1, 1);
```

一般链

```
// 在线段树上对链 (u,v) 上所有点加 val
void updatePath(int u, int v, int val) {
    while (top[u] != top[v]) {
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        // 保证 top[u] 在更深处
        T.update(1, dfn[top[u]], dfn[u], val);
        u = fa[top[u]]; // 跳到上一个链
    }
    // 最后 u,v 在同一条重链上
    if (dep[u] > dep[v]) swap(u, v);
    T.update(1, dfn[u], dfn[v], val);
}
```

链的一端为根的链

```
int st = x;
int ed = top[x];
int res = 0;
while(1){
    res += T.query(1, dfn[ed], dfn[st]);
    if(ed == 1) break;
    st = fa[ed];
    ed = top[st];
}
```

dfs序

扩展dfs序

树上的RMQ与LCA

拓展dfs序：LCA → RMQ

笛卡尔树： RMQ → LCA

欧拉序（常用于树上莫队，树链求异或和）



图论

并查集（路径压缩）

并查集（按秩合并）

```
struct dsu {
  vector<size_t> pa, size;

  explicit dsu(size_t size_) : pa(size_), size(size_, 1) {
    iota(pa.begin(), pa.end(), 0);
  }

  void unite(size_t x, size_t y) {
    x = find(x), y = find(y);
    if (x == y) return;
    if (size[x] < size[y]) swap(x, y);
    pa[y] = x;
    size[x] += size[y];
  }
};
```

带权并查集

```
int find(int x) {
    if(fa[x] == x) return x;
    else {
        int fn = find(fa[x]) ;
        dis[x] += dis[fa[x]];
        return fa[x] = fn;
    }
}
void merge(int x, int y) {
    x = find(x) ; y = find(y);
    if(x != y) {
        fa[x] = y;
        dis[x] = sz[y];
        sz[y] = sz[y] + sz[x];
    }
}
int main()
{
    int m ; cin >> m ;
    for(int i = 1 ; i <= 30000; i++) {
        fa[i] = i ; dis[i] = 0; sz[i] = 1;
    }
    while(m--) {
        char z ;   int x ,y;
        cin >> z >> x  >> y;
        if(z == 'M' ) merge(x , y) ;
        else {
            if(find(x) == find(y) ) cout << abs(dis[x] - dis[y] ) - 1  << endl ;
            else cout << -1 << endl;
        }
    }
    return 0;
```

```
}
```

最小生成树

性质：对于图上的一个环，一定不选最大的那条边。

整个图一定选择最小的能形成树的边。

算法基于比较的方法，也可以适用于最大生成树

Kruskal

//对边排序， 用并查集

kruskal 重构树

洛谷P1967求一张图中两点路径中最小边权的最大值

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e4 + 10;
vector<int> edge2[2 * N];
vector<int> a(2 * N);
const int LOGN = 20, MAXN = 2e4 + 20;
int fa[MAXN][LOGN]; //kruskal
int fa2[MAXN]; //dsu
int find(int x){
    if(x == fa2[x]){
        return x;
    }else {
        return fa2[x]= find(fa2[x]);
    }
}
void merge(int u, int v){
    u = find(u) , v= find(v);
    if(u != v){
        fa2[u] = v;
    }
}
int depth[MAXN];

// 深度优先搜索，预处理 depth 和 fa[u][0]
void dfs(int u, int p, int d) {
    depth[u] = d;
    fa[u][0] = p;
    for (int v : edge2[u]) {
        if (v != p) {
            dfs(v,u, d + 1);
            fa[v][0] = u;
        }
    }
}
// 递推计算 fa[u][k]
void init(int n) {
    for (int k = 1; k < LOGN; ++k) {
        for (int i = 1; i <= n; ++i) {
            if (fa[i][k - 1] != -1) {
```

```
                fa[i][k] = fa[fa[i][k - 1]][k - 1];
            } else {
                fa[i][k] = -1;
            }
        }
    }
}

// 查询 LCA
int lca(int u, int v) {
    // 1. 保证 u 的深度不小于 v
    if (depth[u] < depth[v]) {
        swap(u, v);
    }
    // 2. 将 u 提升到和 v 相同的深度
    for (int k = LOGN - 1; k >= 0; --k) {
        if (depth[u] - (1 << k) >= depth[v]) {
            u = fa[u][k];
        }
    }
    // 3. 如果此时 u 和 v 相等，则 v (或 u) 就是 LCA
    if (u == v) {
        return u;
    }
    // 4. u 和 v 一起向上跳，直到它们的父节点相同
    for (int k = LOGN - 1; k >= 0; --k) {
        if (fa[u][k] != 0 && fa[u][k] != fa[v][k]) {
            u = fa[u][k];
            v = fa[v][k];
        }
    }
    // 5. 返回它们的直接父节点
    return fa[u][0];
}
signed main(){
    int n, m; cin >> n >> m;
    vector<vector<int>> edge(m + 1, vector<int>(3));
    for(int i = 1; i <= m; i++){
        int x, y, z;
        cin >> x >> y >> z;
        edge[i] = {z, x, y};
    }
    for(int i = 1; i <= 2 * n ; i++) fa2[i] = i;
    int cnt = n + 1;
    sort(edge.begin() + 1, edge.begin() + 1 + m, greater<vector<int>>());
    for(int i = 1; i <= m; i++){
        int w = edge[i][0];
        int x = edge[i][1];
        int y = edge[i][2];
        //创建父节点与两点连边，边权为父节点的点券
        if(find(x) != find(y)) {
            edge2[cnt].push_back(find(x));
            edge2[cnt].push_back(find(y));
            edge2[find(x)].push_back(cnt);
            edge2[find(y)].push_back(cnt);
            merge(x, y);
            a[cnt] = w;
            fa2[find(x)] = find(cnt);
            fa2[find(x)] = find(cnt);
```

```
            cnt++;
        }
    }
    dfs(find(1),-1,0);
    for(int i = 2; i <= 2 * n; i++){
        if(find(i) != find(i -1)){
            dfs(find(i),-1,0);
        }
    }
    init(2 * n);
    int q; cin >> q;
    while(q--){
        int x, y; cin >> x >> y;
        if(find(x) != find(y)) cout << -1 << endl;
        else cout << a[lca(x, y)]<< endl;
    }
}
```

Dijkstra

```
     vector<pair<int,int>> edge[N];    // 第一个记录点，第二个记录边权，实数第二个改成double
     vector<int> dis(n + 1);
    void dijstra(int s) {
        for(int i = 1; i <= n; i++) {
            dis[i] = 1e9;
          vis[i] = 0;
        }
        dis[s] = 0;
        q.push({0, s});
        while(!q.empty()) {
            auto [dist, u] = q.top(); q.pop();
            if(vis[u]) continue;
            vis[u] = 1;
            for(auto [v, w]: edge[u]) {
                if(dis[v] > dis[u] + w) {
                    dis[v] = dis[u] + w;
                    pre[v] = u;
                    q.push({-dis[v], v});
                }
            }
        }
    }

    void dfs_path(int u, int s) {
        if(u == s ) cout << u << " ";
        dfs_path(pre[u],  s);
        cout << u <<" ";
    }
```

floyd

```
const int N = 501; // 设定一个足够大的节点数上限
const int INF = 1e9; // 定义无穷大

vector<pair<int, int>> edge[N];
int f[N][N];
```

```
void floyd(int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == j) {
                f[i][j] = 0;
            } else {
                f[i][j] = INF;
            }
        }
    }
    // 从边列表填充初始权重
    for (int i = 1; i <= n; i++) {
        for (auto const& [X, w] : edge[i]) {
            // 防止重边，只取权重最小的边
            f[i][X] = min(f[i][X], w);
        }
    }
    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                // 防止溢出，并且只有当路径存在时才更新
                if (f[i][k] != INF && f[k][j] != INF) {
                    f[i][j] = min(f[i][j], f[i][k] + f[k][j]);
                }
            }
        }
    }
}
```

Bellman-Ford

单源最短路，可以有负权边

可以理解为*bfs*和*dp*，第 *i* 轮可以保证距离原点 *i*的答案是对的

```
bool bellmanford(int n, int s) {
    for(int i = 1; i <= n; i++){
        dis[i] = 1e9;
    }
    dis[s] = 0;
    bool flag = false;  // 判断一轮循环过程中是否发生松弛操作
    for (int i = 1; i <= n; i++) {
        flag = false;
        for (int j = 0; j < edge.size(); j++) {
            u = edge[j].u, v = edge[j].v, w = edge[j].w;
            if (dis[u] == INF) continue;
            // 无穷大与常数加减仍然为无穷大
            // 因此最短路长度为 INF 的点引出的边不可能发生松弛操作
            if (dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;
                flag = true;
            }
        }
        // 没有可以松弛的边时就停止算法
        if (!flag) {
            break;
        }
    }
    // 第 n 轮循环仍然可以松弛时说明 s 点可以抵达一个负环
```

```
    return flag;
}
```

1.二分图的必要条件：没有奇环

2.静态二分图的判定：染色

动态（不断加边）二分图判定：带权并查集/扩展域并查集

Kuhn匈牙利算法

```
struct Kuhn {
    int n, m;
    vector<vector<int>> g;
    vector<int> matchV, vis_tag; // matchV[v] = u
    int vist = 1;

    Kuhn(int n_, int m_) : n(n_), m(m_), g(n_), matchV(m_, -1), vis_tag(m_, 0) {}

    void add_edge(int u, int v) { g[u].push_back(v); }

    bool dfs(int u) {
        for (int v : g[u]) {
            if (vis_tag[v] == vist) continue;
            vis_tag[v] = vist;
            if (matchV[v] == -1 || dfs(matchV[v])) {
                matchV[v] = u;
                return true;
            }
        }
        return false;
    }

    int max_matching() {
        int ans = 0;
        for (int u = 0; u < n; ++u) {
            ++vist;                 // 每次尝试新的增广，把访问标记版本号+1
            if (dfs(u)) ++ans;
        }
        return ans;
    }
};
```

计算几何

基本

```
const double PI = acos(-1.0);
const double EPS = 1e-6; // 精度误差，用于浮点数比较

// --- 浮点数比较函数 ---
// d > 0: 返回 1
// d < 0: 返回 -1
// d = 0: 返回 0
int dcmp(double d) {
    if (std::fabs(d) < EPS) {
        return 0;
    }
    return d < 0 ? -1 : 1;
}

// --- 核心数据结构：点/向量 (Point/Vector) ---
struct Point {
    double x, y;
    // --- 构造函数 ---
    Point(double x = 0.0, double y = 0.0) : x(x), y(y) {}
    // --- 运算符重载 ---
    Point operator+(const Point& b) const {
        return Point(x + b.x, y + b.y);
    }
    Point operator-(const Point& b) const {
        return Point(x - b.x, y - b.y);
    }
    //一定要加
    Point operator*(double k) const {
        return Point(x * k, y * k);
    }

    Point operator/(double k) const {
        return Point(x / k, y / k);
    }

    // 点积 (Dot Product)
    int operator*(const Point& b) const {
        return x * b.x + y * b.y;
    }

    // 叉积 (Cross Product) - 注意：其运算符优先级低于加减法
    //开long long!
    int operator^(const Point& b) const {
        return x * b.y - y * b.x;
    }

    bool operator==(const Point& b) const {
        return dcmp(x - b.x) == 0 && dcmp(y - b.y) == 0;
    }

    // 用于 sort 排序，定义偏序关系
    bool operator<(const Point& b) const {
        int d = dcmp(x - b.x);
        if (d != 0) return d < 0;
```

```
        return dcmp(y - b.y) < 0;
    }
    // --- 几何运算方法 ---
    double len() const {
        return sqrt(x * x + y * y);
    }

    double len2() const {
        return x * x + y * y;
    }

    double dist(const Point& other) const {
        return (*this - other).len();
    }

    // 逆时针转 尽量不要使用三角函数，容易造成精度损失
    Point rot(double rad) const {
        double c = cos(rad), s = sin(rad);
        return Point(x * c - y * s, x * s + y * c);
    }

    Point rot(double cosr, double sinr) const {
        return Point(x * cosr - y * sinr, x * sinr + y * cosr);
    }

     //共起点向量的to-left测试
    int toleft(const Point& a) const {
        int res = (*this) ^ a;
        //return dcmp(res);
        if(res > 0) return 1;
        if(res == 0)return 0;
        if(res < 0) return -1;
    }

    int quad() const // 象限判断 0:原点 1:x轴正 2:第一象限 3:y轴正 4:第二象限 5:x轴负 6:第三象限 7:y轴负 8:第四
象限
    {
        if (abs(x)<=eps && abs(y)<=eps) return 0;
        if (abs(y)<=eps) return x>eps ? 1 : 5;
        if (abs(x)<=eps) return y>eps ? 3 : 7;
        return y>eps ? (x>eps ? 2 : 4) : (x>eps ? 8 : 6);
    }
};


struct Line {
    Point p, v;
    int toleft(const Point& a) const { return v.toleft(a - p); } //toleft测试
    //两直线交点
    //注意共线时(v ^ b.v)为0，返回nan
    //注意重载数乘和加法运算
    Point inter(const Line& b) const {
        Point u = p - b.p;
        double t = (double)(b.v ^ u) / (double)(v ^ b.v);
        return p + v * t;
    }
    double dis(const Point& a) const { return abs(v ^ (a - p)) / v.len(); }//点到直线距离
    Point proj(const Point& a) const { return p + v * ((v * (a - p)) / v.len2()); }//点在直线上的投影
    bool operator<(const line &a) const  // 半平面交算法定义的排序
```

```
    {
        if (abs(v ^ a.v) <= eps && v * a.v >= -eps) return toleft(a.p) == -1;
        return argcmp()(v, a.v);
    }
};

struct Segment {
    Point a, b; // 两个端点
    // -1 点在线段端点 | 0 点不在线段上 | 1 点严格在线段上
    int is_on(const Point& p) const {
        if (p == a || p == b) return -1;
        return dcmp((p - a) ^ (b - a)) == 0 && dcmp((p - a) * (p - b)) < 0;
    }
    // 判断线段直线是否相交
    // -1 直线经过线段端点 | 0 线段和直线不相交 | 1 线段和直线严格相交
    int is_inter(const Line& l) const {
        if (l.toleft(a) == 0 || l.toleft(b) == 0) return -1;
        return l.toleft(a) != l.toleft(b);
    }
    // 判断两线段是否相交
    // -1 在某一线段端点处相交 | 0 两线段不相交 | 1 两线段严格相交
    int is_inter(const Segment& s) const {
        if (is_on(s.a) || is_on(s.b) || s.is_on(a) || s.is_on(b)) return -1;
        Line l1(a, b - a), l2(s.a, s.b - s.a);
        return l1.toleft(s.a) * l1.toleft(s.b) == -1 && l2.toleft(a) * l2.toleft(b) == -1;
    }
    // 点到线段距离（必须用浮点数）
    double dis(const Point& p) const {
        if (dcmp((p - a) * (b - a)) < 0) return p.dist(a);
        if (dcmp((p - b) * (a - b)) < 0) return p.dist(b);
        return Line(a, b - a).dis(p);
    }
    // 两线段间距离（必须用浮点数）
    double dis(const Segment& s) const {
        if (is_inter(s) ) return 0;
        return min({dis(s.a), dis(s.b), s.dis(a), s.dis(b)});
    }
};
```

极角排序

```
struct Argcmp
{
    bool operator()(const Point &a,const Point &b) const
    {
        const int qa=a.quad(),qb=b.quad();
        if (qa!=qb) return qa<qb;
        const auto t=a^b;
        // if (abs(t)<=eps) return a*a<b*b-eps;  // 不同长度的向量需要分开
        return t>eps;
    }
};
```

旋转卡壳

```
T rotcaliper() const
```

```
{
    const auto &p=this->p;
    if (p.size()==1) return 0;
    if (p.size()==2) return p[0].dis2(p[1]);
    const auto area=[](const Point &u,const Point &v,const Point &w){return (w-u)^(w-v);};
    T ans=0;
    for (size_t i=0,j=1;i<p.size();i++)
    {
        const auto nxti=this->nxt(i);
        ans=max({ans,p[j].dis2(p[i]),p[j].dis2(p[nxti])});
        while (area(p[this->nxt(j)],p[i],p[nxti])>=area(p[j],p[i],p[nxti]))
        {
            j=this->nxt(j);
            ans=max({ans,p[j].dis2(p[i]),p[j].dis2(p[nxti])});
        }
    }
    return ans;
}
```

凸包 (Convex Hull)

```
// Andrew求凸包
// 返回一个 Convex 对象，其中包含按逆时针顺序排列的凸包顶点
vector<Point> convex_hull(vector<Point> p) {
    // 按 x, y 坐标排序
    sort(p.begin(), p.end());
    // 删除重复点
    p.erase(unique(p.begin(), p.end()), p.end());

    if (p.size() <= 2) return p;

    vector<Point> st; // 用于构造凸包的栈

    // 检查下一个点 u 是否在前两个点的右侧（或共线），若是则出栈
    const auto check = [](const vector<Point> &st, const Point &u) {
        const auto back1 = st.back();
        const auto back2 = *prev(st.end(), 2);
        // (back1 - back2) 是栈顶的边向量
        // toleft <= 0 表示 u 在这条边的右侧或共线，需要pop
        return (back1 - back2).toleft(u - back1) <= 0;
    };

    // 构造下凸包
    for (const Point &u : p) {
        while (st.size() > 1 && check(st, u)) {
            st.pop_back();
        }
        st.push_back(u);
    }

    size_t k = st.size(); // 下凸包的大小
    // 构造上凸包
    // 从倒数第二个点开始（因为最后一个点已经在下凸包里了）
    for (int i = p.size() - 2; i >= 0; --i) {
        const Point &u = p[i];
        while (st.size() > k && check(st, u)) {
            st.pop_back();
        }
```

```
        st.push_back(u);
    }
    st.pop_back(); // 移除重复的起始点
    return st;
}
```

凸包

```
vector<Point> convex_hull(vector<Point> p){
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()),p.end());
    if(p.size() <= 2) return p;
    vector<Point> st;
    auto check = [](const vector<Point> &st, const Point &u){
        auto back1 = st.back();
        auto back2 = *prev(st.end(), 2);
        return (back1 - back2).toleft(u - back1) <= 0;
    };
    for(const Point &u :p){
        while(st.size() > 1 && check(st, u)){
            st.pop_back();
        }
        st.push_back(u);
    }
    int k = st.size();
    for(int i = p.size() - 2; i >= 0; i--){
        Point &u = p[i];
        while(st.size() > k && check(st,u)){
            st.pop_back();
        }
        st.push_back(u);
    }
    st.pop_back();
    return st;
}
```

半平面交

```
// 半平面交
// 排序增量法，复杂度 O(nlogn)
// 输入与返回值都是用直线表示的半平面集合
//有向直线左侧是合法区域
//按方向角排序，同向只留最严格的
//新线切掉队尾顶点：弹尾
//新线切掉队首顶点：弹头
//全部加入后再进行首尾封口

vector<Line> halfinter(vector<Line> l, const point_t lim = 1e9) {
    const auto check = [](const Line &a, const Line &b, const Line &c) {
        return a.toleft(b.inter(c)) < 0;
    };
    // 无精度误差的方法，但注意取值范围会扩大到三次方
    /*const auto check=[](const Line &a,const Line &b,const Line &c)
    {
        const Point
    p=a.v*(b.v^c.v),q=b.p*(b.v^c.v)+b.v*(c.v^(b.p-c.p))-a.p*(b.v^c.v); return
```

```
    p.toleft(q)<0;
    };*/
    l.push_back({{-lim, 0}, {0, -1}});
    l.push_back({{0, -lim}, {1, 0}});
    l.push_back({{lim, 0}, {0, 1}});
    l.push_back({{0, lim}, {-1, 0}});
    sort(l.begin(), l.end());
    deque<Line> q;
    for (size_t i = 0; i < l.size(); i++) {
        if (i > 0 && l[i - 1].v.toleft(l[i].v) == 0 &&
            l[i - 1].v * l[i].v > eps)
            continue;
        while (q.size() > 1 && check(l[i], q.back(), q[q.size() - 2]))
            q.pop_back();
        while (q.size() > 1 && check(l[i], q[0], q[1])) q.pop_front();
        if (!q.empty() && q.back().v.toleft(l[i].v) <= 0) return vector<Line>();
        q.push_back(l[i]);
    }
    while (q.size() > 1 && check(q[0], q.back(), q[q.size() - 2])) q.pop_back();
    while (q.size() > 1 && check(q.back(), q[0], q[1])) q.pop_front();
    return vector<Line>(q.begin(), q.end());
}
```

凸多边形

```
// 凸多边形
template <typename T>
struct convex : polygon<T> {
    // 闵可夫斯基和
    convex operator+(const convex &c) const {
        const auto &p = this->p;
        vector<Segment> e1(p.size()), e2(c.p.size()),
            edge(p.size() + c.p.size());
        vector<point<T>> res;
        res.reserve(p.size() + c.p.size());
        const auto cmp = [](const Segment &u, const Segment &v) {
            return argcmp()(u.b - u.a, v.b - v.a);
        };
        for (size_t i = 0; i < p.size(); i++) e1[i] = {p[i], p[this->nxt(i)]};
        for (size_t i = 0; i < c.p.size(); i++) e2[i] = {c.p[i], c.p[c.nxt(i)]};
        rotate(e1.begin(), min_element(e1.begin(), e1.end(), cmp), e1.end());
        rotate(e2.begin(), min_element(e2.begin(), e2.end(), cmp), e2.end());
        merge(e1.begin(), e1.end(), e2.begin(), e2.end(), edge.begin(), cmp);
        const auto check = [](const vector<point<T>> &res, const point<T> &u) {
            const auto back1 = res.back(), back2 = *prev(res.end(), 2);
            return (back1 - back2).toleft(u - back1) == 0 &&
                   (back1 - back2) * (u - back1) >= -eps;
        };
        auto u = e1[0].a + e2[0].a;
        for (const auto &v : edge) {
            while (res.size() > 1 && check(res, u)) res.pop_back();
            res.push_back(u);
            u = u + v.b - v.a;
        }
        if (res.size() > 1 && check(res, res[0])) res.pop_back();
        return {res};
    }
```

```
    // 旋转卡壳
    // func 为更新答案的函数，可以根据题目调整位置
    template <typename F>
    void rotcaliper(const F &func) const {
        const auto &p = this->p;
        const auto area = [](const point<T> &u, const point<T> &v,
                             const point<T> &w) { return (w - u) ^ (w - v); };
        for (size_t i = 0, j = 1; i < p.size(); i++) {
            const auto nxti = this->nxt(i);
            func(p[i], p[nxti], p[j]);
            while (area(p[this->nxt(j)], p[i], p[nxti]) >=
                   area(p[j], p[i], p[nxti])) {
                j = this->nxt(j);
                func(p[i], p[nxti], p[j]);
            }
        }
    }

    // 凸多边形的直径的平方
    T diameter2() const {
        const auto &p = this->p;
        if (p.size() == 1) return 0;
        if (p.size() == 2) return p[0].dis2(p[1]);
        T ans = 0;
        auto func = [&](const point<T> &u, const point<T> &v,
                        const point<T> &w) {
            ans = max({ans, w.dis2(u), w.dis2(v)});
        };
        rotcaliper(func);
        return ans;
    }

    // 判断点是否在凸多边形内
    // 复杂度 O(logn)
    // -1 点在多边形边上 | 0 点在多边形外 | 1 点在多边形内
    int is_in(const point<T> &a) const {
        const auto &p = this->p;
        if (p.size() == 1) return a == p[0] ? -1 : 0;
        if (p.size() == 2) return segment<T>{p[0], p[1]}.is_on(a) ? -1 : 0;
        if (a == p[0]) return -1;
        if ((p[1] - p[0]).toleft(a - p[0]) == -1 ||
            (p.back() - p[0]).toleft(a - p[0]) == 1)
            return 0;
        const auto cmp = [&](const Point &u, const Point &v) {
            return (u - p[0]).toleft(v - p[0]) == 1;
        };
        const size_t i =
            lower_bound(p.begin() + 1, p.end(), a, cmp) - p.begin();
        if (i == 1) return segment<T>{p[0], p[i]}.is_on(a) ? -1 : 0;
        if (i == p.size() - 1 && segment<T>{p[0], p[i]}.is_on(a)) return -1;
        if (segment<T>{p[i - 1], p[i]}.is_on(a)) return -1;
        return (p[i] - p[i - 1]).toleft(a - p[i - 1]) > 0;
    }

    // 凸多边形关于某一方向的极点
    // 复杂度 O(logn)
    // 参考资料：https://codeforces.com/blog/entry/48868
    template <typename F>
    size_t extreme(const F &dir) const {
```

```
        const auto &p = this->p;
        const auto check = [&](const size_t i) {
            return dir(p[i]).toleft(p[this->nxt(i)] - p[i]) >= 0;
        };
        const auto dir0 = dir(p[0]);
        const auto check0 = check(0);
        if (!check0 && check(p.size() - 1)) return 0;
        const auto cmp = [&](const Point &v) {
            const size_t vi = &v - p.data();
            if (vi == 0) return 1;
            const auto checkv = check(vi);
            const auto t = dir0.toleft(v - p[0]);
            if (vi == 1 && checkv == check0 && t == 0) return 1;
            return checkv ^ (checkv == check0 && t <= 0);
        };
        return partition_point(p.begin(), p.end(), cmp) - p.begin();
    }

    // 过凸多边形外一点求凸多边形的切线，返回切点下标
    // 复杂度 O(logn)
    // 必须保证点在多边形外
    pair<size_t, size_t> tangent(const point<T> &a) const {
        const size_t i = extreme([&](const point<T> &u) { return u - a; });
        const size_t j = extreme([&](const point<T> &u) { return a - u; });
        return {i, j};
    }

    // 求平行于给定直线的凸多边形的切线，返回切点下标
    // 复杂度 O(logn)
    pair<size_t, size_t> tangent(const line<T> &a) const {
        const size_t i = extreme([&](...) { return a.v; });
        const size_t j = extreme([&](...) { return -a.v; });
        return {i, j};
    }
};

using Convex = convex<point_t>;
```

字符串

字符串哈希

```
using ull = unsigned long long;
ull base = 131, mod1 = 212370440130137957, mod2 = 1e9 + 7;
ull get_hash1(std::string s) {
    ull ans = 0,len=s.size();
    for (int i = 0; i < len; i++) ans = (ans * base + (ull)s[i]) % mod1;
    return ans;
}
ull get_hash2(std::string s) {
    ull ans = 0,len=s.size();
    for (int i = 0; i < len; i++) ans = (ans * base + (ull)s[i]) % mod2;
    return ans;
}
bool cmp(const std::string s, const std::string t) {
    return get_hash1(s) != get_hash1(t) || get_hash2(s) != get_hash2(t);
}
```

字典树

```
const int N = 3e6 + 50 ;
struct trie{
    int idx = 0, son[N][65] , cnt[N];

    void init(){
        for(int i = 0; i <= idx; i++){
            for(int j = 0; j < 65; j++){
                son[i][j] = 0;
            }
        }

        for(int i = 0; i <= idx; i++) cnt[i] = 0;
        idx = 0;
        return;
    }

    int getnum(char x){
        if(x >= 'A' && x <= 'Z') return x - 'A';
        else if(x >= 'a' && x <= 'z') return x - 'a' + 26;
        else return x - '0' + 52;
    }
    void insert(string s){
        int p = 0;
        for(auto c : s){
            int x = getnum(c);
            if(!son[p][x]) son[p][x] = ++idx;
            p = son[p][x];
            cnt[p]++;
        }
        return;
    }
    int query(string s){
        int p = 0;
        for(auto c : s){
```

```
            int x = getnum(c);
            if(!son[p][x]) return 0;
            p = son[p][x];
        }
        return cnt[p];
    }
}trie;

void solve(){
    int n, q; cin >> n >> q;
    trie.init();
    for(int i = 1;  i <= n; i++){
        string s; cin >> s;
        trie.insert(s);
    }
    while(q--){
        string s;
        cin >> s;
        cout << trie.query(s) << endl;
    }
    return;
}
```

KMP

Manacher

AC自动机

dp

SOSdp（高维前缀和）

子集和dp

```
// SOS DP 模板
// f[mask] 初值为每个集合 mask 的信息
// 经过处理后，f[mask] = 所有子集的和

for(int i = 0; i < k; i++){
    for(int mask = 0; mask < (1 << k); mask++){
        if(mask & (1 << i)){
            f[mask] += f[mask ^ (1 << i)];
        }
    }
}
```

超集和dp

```
for(int i = 0; i < k; i++){
    for(int mask = 0; mask < (1<<k); mask++){
        if((mask & (1<<i)) == 0){
            f[mask] += f[mask ^ (1<<i)];
        }
    }
}
```

二进制下枚举子集

复杂度 $O(3^k)$

1008 cats 的 max

设有 `k` 个元素，全集掩码 `ALL = (1<<k) - 1`。
常用模式与模板如下（**均为闭式枚举，含 0 子集**）。

1. 枚举 `mask` 的所有子集 `sub`（含 0）

```
// 枚举 mask 的所有子集 sub（含 0）
for (int sub = mask; ; sub = (sub - 1) & mask) {
    // ... 使用 sub ...
    if (sub == 0) break;
}
```

2. 枚举与 `mask` **不相交** 的子集（即补集上的子集）

```
int rest = ALL ^ mask; // 补集（剩余可用位）
for (int sub = rest; ; sub = (sub - 1) & rest) {
    // sub 与 mask 不相交，且 sub ⊆ rest
    if (sub == 0) break;
}
```

3. 枚举把 `mask` 拆成两部分：`sub` 与 `mask^sub`

```
// 遍历 mask 的所有划分 (sub, mask^sub)
for (int sub = mask; ; sub = (sub - 1) & mask) {
    int other = mask ^ sub;
    // ... 使用 (sub, other) ...
    if (sub == 0) break;
}
```

树上背包 $(O(n^2))$

Problem - I - Codeforces

每个点对只被选到一次

```
     for (auto X: edge[x]){
         if (X == f) continue;
         dfs(X, x);
         for (int j = 0; j <= sz[X]; j++) {
             for (int k = 0; k <= sz[x]; k++) {
                 j和k做卷积
                   dp[j + k] <- dp[j], dp[k]
             }
         }
         sz[x] += sz[X]; //注意一定要写在for的后面
     }
```

for循环的复杂度事实上贡献自`x`和`x`两个点堆内的点的配对。我们现在考虑树内所有点的配对。(u,v)只在$lca(u,v)$这个点处被枚举一次。因此树上背包的复杂度为 $O(n^2)$

01背包

时间复杂度O(nm), 空间O(m)

```
for i=1..N
    for v=V..0
        f[v]=max{f[v],f[v-c[i]]+w[i]};
```

完全背包

时间复杂度O(nm), 空间O(m)

```
for i=1..N
    for v=0..V
        f[v]=max{f[v],f[v-c[i]]+w[i]};
```

多重背包

与 0-1 背包的区别在于每种物品有k_i 个，而非一个。

Naive Algorithm

把每种物品复制为k_i个，转化为01背包

单调队列优化O(nm)

数学

exgcd

```
/**
 * @brief 拓展欧几里得算法
 * @param a 第一个整数
 * @param b 第二个整数
 * @param x 方程 ax + by = gcd(a, b) 的一个解
 * @param y 方程 ax + by = gcd(a, b) 的一个解
 * @return 返回 a 和 b 的最大公约数
 */

ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll gcd = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return gcd;
}
```

线性筛

```
const int N = 1e6 + 10;
vector<int> prime; //素数表, prime[0] = 2, prime[1] = 3, prime[2] = 5, ...
vector<int> pm(N); //记录i的最小质数 pm[2] = 2, pm[3] = 3, pm[4] = 2, pm[5] = 5, pm[6] = 2, pm[7] = 7,
...
vector<int> isprime(N);

// 使用欧拉筛法生成素数表
// 对于每一个合数n, 欧拉筛在i = n/pm[n], prime[j] = pm[n]时筛去n
void seive(int n){
    int tot = 0;
    vector<int> vis(n + 1);
    for(int i = 2; i <= n; i++){
        if(!vis[i])  {
            prime.push_back(i);
            isprime[i] = 1;
            pm[i] = i;
        }
        for(int j = 0; j < prime.size() && i * prime[j] <= n ; j++){
            vis[i * prime[j]] = 1;
            pm[i * prime[j]] = prime[j]; //记录 i*prime[j] 的最小素因子
            if(i % prime[j] == 0) break;
        }
    }
}
```

积性函数

```
ll get_f(ll n){
    ll ans = 1;
    for(int i = 2; i <= n/ i; i++){
        int cnt = 0;
        while(n % i == 0) cnt++, n/= i;
        ans *= f(i, cnt);  // f(p,k)= f(p^k)
    }
    if(n > 1) ans *= f(n, 1);
    return ans;
}
```

莫比乌斯反演

1. 莫比乌斯函数的定义

$$\mu(n) = \begin{cases} 1 & n = 1, \\ (-1)^r & n \text{ 是 } r \text{ 个不同质数的积}, \\ 0 & n \text{ 含有平方因子}. \end{cases} \tag{1}$$

等价地，$\mu(n)$ 是积性函数，且对质数 p：

$$\mu(p) = -1, \quad \mu(p^k) = 0 \ (k \geq 2). \tag{2}$$

---

2. 莫比乌斯变换与反演公式

(1) 狄利克雷卷积形式

- 卷积定义：

$$(f * g)(n) = \sum_{d|n} f(d)\, g\left(\tfrac{n}{d}\right). \tag{3}$$

- 单位元：

$$\varepsilon(n) = \begin{cases} 1 & n = 1, \\ 0 & n > 1. \end{cases} \tag{4}$$

---

(2) 正变换

若

$$g(n) = \sum_{d|n} f(d), \tag{5}$$

即

$$g = f * 1. \tag{6}$$

---

(3) 逆变换

因为 $\mu * 1 = \varepsilon$，所以

$$f = g * \mu, \tag{7}$$

即

$$f(n) = \sum_{d \mid n} \mu(d)\, g\!\left(\tfrac{n}{d}\right). \tag{8}$$

---

3. 常见应用

(0) ε的展开

$$\sum_{d \mid n} \mu(d) = [n = 1]. \tag{9}$$

(1) 因子和反演

$$g(n) = \sum_{d \mid n} f(d) \quad \Longleftrightarrow \quad f(n) = \sum_{d \mid n} \mu(d)\, g\!\left(\tfrac{n}{d}\right). \tag{10}$$

(2) 卷积单位元表示

$$\varepsilon = \mu * 1. \tag{11}$$

即

$$\sum_{d \mid n} \mu(d) = \begin{cases} 1 & n = 1, \\ 0 & n > 1. \end{cases} \tag{12}$$

(3) 欧拉函数恒等式

$$\varphi = \mu * \mathrm{id}, \quad \varphi(n) = \sum_{d \mid n} \mu(d)\, \frac{n}{d}. \tag{13}$$

(4) 计数互素对

若

$$F(n) = \sum_{i=1}^{n} \sum_{j=1}^{n} [\gcd(i, j) = 1], \tag{14}$$

则可通过莫比乌斯反演化简。

- **定义**：$\mu(n)$ 在质数幂下的取值。
- **正变换**：$g(n) = \sum_{d \mid n} f(d)$。
- **逆变换**：$f(n) = \sum_{d \mid n} \mu(d) g(n/d)$。
- **重要关系**：$\mu * 1 = \varepsilon$，$\varphi = \mu * \mathrm{id}$。

常见莫比乌斯反演恒等式

1. 基础恒等式

- **卷积单位**

$$\mu * 1 = \varepsilon, \quad \sum_{d \mid n} \mu(d) = [n = 1]. \tag{15}$$

- **欧拉函数**

$$\varphi = \mu * \mathrm{id}, \quad \varphi(n) = \sum_{d|n} \mu(d)\frac{n}{d}. \tag{16}$$

- **约数个数函数与约数和函数**

$$d(n) = \sum_{d|n} 1, \quad \sigma(n) = \sum_{d|n} d = (1 * id)(n). \tag{17}$$

2. 反演恒等式

- **狄利克雷反演**
  若

$$g(n) = \sum_{d|n} f(d), \tag{18}$$

则

$$f(n) = \sum_{d|n} \mu(d)\, g\big(\tfrac{n}{d}\big). \tag{19}$$

---

3. 常见和式公式

- **欧拉函数求和**

$$\sum_{d|n} \varphi(d) = n. \tag{20}$$

- **莫比乌斯和整除分块**

$$\sum_{d=1}^{n} \mu(d)\ \left\lfloor \tfrac{n}{d} \right\rfloor = 1. \tag{21}$$

- **互素计数**

$$\sum_{i=1}^{n}\sum_{j=1}^{n} [\gcd(i,j)=1] = \sum_{d=1}^{n} \mu(d)\ \left\lfloor \tfrac{n}{d} \right\rfloor^2. \tag{22}$$

- **一般互素 k 元组**

$$\#\{(x_1,\dots,x_k) : 1 \le x_i \le n, \gcd(x_1,\dots,x_k)=1\} = \sum_{d=1}^{n} \mu(d)\ \left\lfloor \tfrac{n}{d} \right\rfloor^k. \tag{23}$$

---

4. 与积性函数关系

- **φ 的母函数**

$$\sum_{d|n} \varphi(d) = n. \tag{24}$$

- **μ 的平方**

$$\mu^2(n) = [n \text{ 无平方因子}]. \tag{25}$$

- **约数和函数与 id 的关系**

$$\sigma_k = id^k * 1. \tag{26}$$

矩阵乘法与矩阵快速幂

```cpp
#include <bits/stdc++.h>
using ll = long long;
const int MOD = 1e9 + 7;
// 矩a阵结构体
struct Matrix {
    int n, m; // n: 行数, m: 列数
    vector<vector<ll>> a;
    // 构造函数, 创建一个 n x m 的零矩阵
    // 使用 (n+1) x (m+1) 的大小来支持1-based索引
    Matrix(int _n, int _m) : n(_n), m(_m), a(_n + 1, std::vector<ll>(_m + 1, 0)) {}
    // 创建一个 n x n 的单位矩阵
    static Matrix identity(int n) {
        Matrix I(n, n);
        for (int i = 1; i <= n; ++i) {
            I.a[i][i] = 1;
        }
        return I;
    }

    // 打印矩阵 (用于调试)
    void print() const {
        for (int i = 1; i <= n; ++i) {
            for (int j = 1; j <= m; ++j) {
                std::cerr << a[i][j] << " ";
            }
            std::cerr << std::endl;
        }
    }
};
// 矩阵乘法: C = A * B
// 时间复杂度: O(n^3) 或 O(A.n * A.m * B.m)
Matrix operator*(const Matrix& A, const Matrix& B) {
    Matrix C(A.n, B.m);
    for (int i = 1; i <= A.n; ++i) {
        for (int j = 1; j <= B.m; ++j) {
            for (int k = 1; k <= A.m; ++k) {
                C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;
            }
        }
    }
    return C;
}

// 矩阵快速幂: A^p, 时间复杂度: O(n^3 * log p) , A必须是方阵
Matrix power(Matrix A, ll p) {
    Matrix res = Matrix::identity(A.n);
    A.a.shrink_to_fit(); // 可选: 在某些情况下可能节省一点内存
    while (p) {
        if (p & 1) res= res * A;
        A = A * A;
        p >>= 1;
```

```
    }
    return res;
}
```

高斯消元

```
int gauss_jordan(vector<vector<double>>& a, int vars) {
    int eqs = a.size();
    if (eqs == 0) return 0;
    int rk = 0;
    for (int col = 0; col < vars && rk < eqs; ++col) {
        int pivot_row = rk;
        for (int i = rk + 1; i < eqs; ++i) {
            if (abs(a[i][col]) > abs(a[pivot_row][col])) {
                pivot_row = i;



            }
        }
        swap(a[rk], a[pivot_row]);
        if (abs(a[rk][col]) < EPS) {
            continue;
        }
        double pivot_val = a[rk][col];
        for (int j = col; j < a[0].size(); ++j) {
            a[rk][j] /= pivot_val;
        }
        for (int i = 0; i < eqs; ++i) {
            if (i != rk) {
                double factor = a[i][col];
                for (int j = col; j < a[0].size(); ++j) {
                    a[i][j] -= factor * a[rk][j];
                }
            }
        }
        rk++;
    }
    return rk;
}
```

线性基 （寻找第k大有些问题)

```
using ll = long long;

const int Maxb = 62; // long long 的最大位数，通常到62就足够
struct LinearBasis {
    ll b[Maxb + 1];
    int cnt; // 基向量的数量
    bool zero_possible; // 原集合是否能异或出0
    LinearBasis() {
        memset(b, 0, sizeof(b));
        cnt = 0;
```

```
        zero_possible = false;
    }
    // 插入一个数 x 时间复杂度：O(log x)
    void insert(ll x) {
        for (int i = Maxb; i >= 0; --i) {
            // 如果 x 的第 i 位是 0，跳过
            if (!(x >> i & 1)) continue;
            if (!b[i]) { // 如果 b[i] 是空的，找到了 x 的位置
                b[i] = x;
                cnt++;
                return;
            }
            // 如果 b[i] 已有值，用它来消去 x 的第 i 位
            x ^= b[i];
        }
        // 如果 x 最终变成了 0，说明 x 可以由之前的基向量表示，这意味着原集合中存在线性相关的数，可以异或出 0
        zero_possible = true;
    }
    // 查询原集合的子集能异或出的最大值,时间复杂度：O(log MAX_VAL)，基于贪心
    ll query_max() const {
        ll res = 0;
        for (int i = Maxb; i >= 0; --i) {
            res = max(res, res ^ b[i]);
        }
        return res;
    }

    // 查询原集合的子集能异或出的最小值（非0） 时间复杂度：O(log MAX_VAL)
    ll query_min() const {
        if (zero_possible) return 0; // 如果能凑出0，最小就是0
        for (int i = 0; i <= Maxb; ++i) {
            if (b[i]) {
                return b[i];
            }
        }
        return 0; // 集合为空
    }
    // 查询 x 是否能被原集合的子集异或出来
    // 时间复杂度：O(log x)
    bool can_form(ll x) const {
        for (int i = Maxb; i >= 0; --i) {
            if (x >> i & 1) {
                x ^= b[i];
            }
        }
        return x == 0;
    }

    // 查询第 k 小的异或和
    // k 从 1 开始计数
    // 时间复杂度：O((log MAX_VAL)^2)
    ll query_kth(ll k) const {
        // 如果能异或出0，那么0就是第1小的。我们查询的k要减1
        if (zero_possible) {
            if (k == 1) return 0;
            k--;
        }
        // 检查k是否越界
        if (k >= (1LL << cnt)) {
```

```
            return -1; // 表示不存在第k小的值
        }
        // 1. 重构基，使其成为一个“三角矩阵”形式，方便计算
        vector<ll> p;
        for (int i = 0; i <= Maxb; ++i) {
            if (b[i]) {
                // 用更高的位消去当前b[i]中对应的位
                for (ll val : p) {
                    b[i] = std::min(b[i], b[i] ^ val);
                }
                p.push_back(b[i]);
                std::sort(p.begin(), p.end());
            }
        }
        // 2. 根据 k 的二进制位来构造答案
        ll res = 0;
        for (size_t i = 0; i < p.size(); ++i) {
            if (k >> i & 1) {
                res ^= p[i];
            }
        }
        return res;
    }
};

// --- 模板使用示例 ---
void run_example() {
    LinearBasis lb;
    std::vector<ll> nums = {3, 5, 6}; // 集合 {011, 101, 110}
    for (ll num : nums) {
        lb.insert(num);
    }
    // 插入 {3, 5, 6} 后，线性基可以是 {b[1]=3, b[2]=6} 或 {b[0]=1, b[1]=2, b[2]=4} 的某种形式
    // 假设是 {b[0]=1, b[1]=2, b[2]=4} (3^5=6, 3^6=5) ... 不对
    // 插入 3 (011): b[1]=3
    // 插入 5 (101): b[2]=5
    // 插入 6 (110): x=6. i=2, x^=b[2]=5 -> x=3. i=1, x^=b[1]=3 -> x=0.
    // 最终线性基: b[2]=5, b[1]=3.
    // cnt=2, zero_possible=true (因为6可以被{3,5}表示)

    std::cout << "Max XOR sum: " << lb.query_max() << std::endl; // 应该是 5^3 = 6
    std::cout << "Min XOR sum: " << lb.query_min() << std::endl; // 能凑出0, 所以是 0

    std::cout << "Can form 2? " << (lb.can_form(2) ? "Yes" : "No") << std::endl; // 3^5=6, 3^6=5,
5^6=3, 无法凑出2
    std::cout << "Can form 6? " << (lb.can_form(6) ? "Yes" : "No") << std::endl; // 3^5=6

    std::cout << "--- K-th smallest XOR sums ---" << std::endl;
    // 可能的异或和: 0, 3, 5, 6
    // 排序后: 0, 3, 5, 6
    std::cout << "1st smallest: " << lb.query_kth(1) << std::endl; // 0
    std::cout << "2nd smallest: " << lb.query_kth(2) << std::endl; // 3
    std::cout << "3rd smallest: " << lb.query_kth(3) << std::endl; // 5
    std::cout << "4th smallest: " << lb.query_kth(4) << std::endl; // 6
    std::cout << "5th smallest: " << lb.query_kth(5) << std::endl; // -1 (不存在)
}

int main() {
    run_example();
```

```
    return 0;
}
```

Luca定理

应用场景

当我们需要计算非常大的组合数 $C(n,m) \bmod p$，其中 n,m 很大而 p 是**质数**，常规的阶乘预处理无法承受时，Lucas 定理提供了一种分解方法。

定理表述

设 `p` 为质数，`n,m` 为非负整数，则有：

$$\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod{p}, \tag{27}$$

其中

- $n = n_k p^k + n_{k-1} p^{k-1} + \cdots + n_0$
- $m = m_k p^k + m_{k-1} p^{k-1} + \cdots + m_0$

为 n,m 在 p 进制下的展开。

特别地，当 $m_i > n_i$ 时，$\binom{n_i}{m_i} = 0$。

证明思路（简要）

- 基于二项式展开和模 `p` 的性质。
- 将 `n,m` 写成 `p` 进制分块，利用模质数时的组合数分解性质得到结论。

实现要点

1. 预处理 $[0, p-1]$ 范围内的阶乘和逆元，用于快速计算小组合数。
2. 对 n,m 做 p 进制分解，逐位计算组合数并累乘。
3. 时间复杂度：$O(\log_p n)$。

模板代码（C++）

```
// p 必须是质数
int C(int n, int m, int p, vector<int> &fac, vector<int> &invfac) {
    if (m < 0 || m > n) return 0;
    return 1LL * fac[n] * invfac[m] % p * invfac[n - m] % p;
}

int Lucas(long long n, long long m, int p, vector<int> &fac, vector<int> &invfac) {
    if (m == 0) return 1;
    return 1LL * Lucas(n / p, m / p, p, fac, invfac) *
           C(n % p, m % p, p, fac, invfac) % p;
}
```

斯特林数

多项式 写在前面

多项式 `f(x) ,g(x)` 系数表示 -> 点值表示 `O(nlogn)`

h(x) = f(x) * g(x) 在点值表示下

--> h(x)系数表示 `O(nlogn)`

DFT(离散傅里叶变换，discrete Fourier transform)

$$将多项式A(x)=a_0+a_1x+\cdots+a_{n-1}x^{n-1}转化为其点值形式(\omega_n^k,A(\omega_n^k)),\quad (k=0,1,\ldots,n-1)。\qquad (28)$$







IDFT

$$将多项式的点值表示(\omega_n^k,b_k),\quad (k=0,1,\ldots,n-1)转化为其系数表示A(x)=a_0+a_1x+\cdots+a_{n-1}x^{n-1}.\quad (29)$$





简洁的fft

```
#include <bits/stdc++.h>
using namespace std;
typedef complex<double> cd;
const double pi = acos(-1);
int tot;
vector<int> r;

void fft(vector<cd> &a,  int op){
    for (int i = 0; i < tot; i++)
        if (i < r[i]) swap(a[i], a[r[i]]);
    for (int m = 2; m <= tot; m <<= 1){
        cd wk1 = {cos(pi * 2 / m), sin(pi * 2 / m) * op};
        for (int i = 0; i < tot; i += m){
            cd wk = {1, 0};
            for (int j = 0; j < m / 2; j++){
                cd x = a[j + i], y = a[j + i + m / 2] * wk;
                a[j + i] = x + y;
                a[j + i + m / 2] = x - y;
                wk = wk * wk1;
            }
        }
    }
}
```

```
signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; cin >> n;
    int m; cin >> m;
    n++;
    m++;
    int bit = 0;
    while((1 << bit) < (n + m + 1)) bit++;
    tot = 1 << bit;
    r.resize(tot);
    for (int i = 0; i < tot; i++)
        r[i] = r[i / 2] / 2 + ((i & 1) ? tot / 2 : 0);

    vector<cd> a(tot), b(tot);
    for(int i = 0; i < n; i++){
        cin >> a[i];
    }
    for(int i = 0; i < m; i++) {
        cin >> b[i];
    }
    fft(a, 1);
    fft(b, 1);
    vector<cd> conv(tot);
    for(int i = 0; i < tot; i++){
        conv[i] = (a[i] * b[i]);
    }
    fft(conv, -1);

    for(int i = 0; i < n + m - 2 + 1; i++){
        cout << (int) (conv[i].real() / tot + 0.5) << " ";
    }
}
```

简洁的ntt

```
#include <bits/stdc++.h>
using namespace std;
using T = long long;
using ll = long long;
const long long mod = 1004535809, G = 3, Gi = 334845270;
int tot;
vector<int> rev;
T fastpow(T a, T b){
    T res = 1;
    while (b){
        if (b & 1) res = (res * a) % mod;
        b >>= 1;
        a = (a * a) % mod;
    }
    return res;
}
T Inv(T x){
    return fastpow(x, mod - 2);
}
```

```
void NTT(vector<T> &c, int inv)
{
    for (int i = 0; i < tot; i++){
        if (i < rev[i]) swap(c[i], c[rev[i]]);
    }
    for (int mid = 1; mid < tot; mid <<= 1){  // 枚举每个子问题的mid
        T w1 = fastpow(G, (mod - 1) / (mid << 1)); // g^{(mod-1)/N}
        if(!(~inv))
            w1 = fastpow(w1, mod - 2); // 如果是逆变换,要在指数上变负号
        for(T i = 0, len = mid << 1; i < tot; i += len){
            T wk = 1;
            for (T j = 0; j < mid; j++, wk = (wk * w1) % mod){ // 处理一半足矣
                T x = c[i + j], y = wk * c[i + j + mid] % mod;
                c[i + j] = (x + y) % mod;
                c[i + j + mid] = (x - y + mod) % mod;
            }
        }
    }
}

signed main()
{
    int n; cin >> n;
    int m; cin >> m;

    int bit = 0;
    while((1 << bit) < (n + m + 1)) bit++;
    tot = 1 << bit;
    rev.resize(tot);
    vector<T> a(tot), b(tot);
    for(int i = 0; i < n; i++){
        ll x; cin >> x;
        a[i] = x;
    }
    for(int i = 0; i < m; i++) {
        ll x; cin >> x;
        b[i] = x;
    }
    for(int i = 0; i< tot;i++){
          rev[i] = ( rev[i>>1] >>1 ) | ( (i & 1) << ( bit - 1) );
     }
    NTT(a, 1);
    NTT(b, 1);
    T _tot = Inv(tot);
    vector<T> conv(tot);
    for(int i = 0; i < tot; i++){
        conv[i] = (a[i] * b[i])  % mod;
    }
    NTT(conv, -1);

    for(int i = 0; i < n + m - 2 + 1; i++){
        cout << (ll) (conv[i] * _tot % mod) << " ";
    }
}
```

常见模数

$$
\begin{aligned}
65537 &= 2^{16}+1, g=3 \\
998244353 &= 119*2^{23}+1, g=3 \\
1004535809 &= 479*2^{21}+1, g=3 \\
4179340454199820289 &= 29*2^{57}+1, g=3 \\
g_i &\to g^{-1}(mod)
\end{aligned} \tag{30}
$$

FWT

```
void fwtor(int a[], int m, int opt) { //(1,-1)
  for (int len = 2; len <= m; len <<= 1)
    for (int p = len >> 1, i = 0; i < m; i += len)
      for (int j = i; j < i + p; j++)
        if (opt > 0)
          add(a[j + p], a[j]);
        else
          del(a[j + p], a[j]);
 }
 void fwtand(int a[], int m, int opt) { //(1,-1)
  for (int len = 2; len <= m; len <<= 1)
    for (int p = len >> 1, i = 0; i < m; i += len)
      for (int j = i; j < i + p; j++)
        if (opt > 0)
          add(a[j], a[j + p]);
        else
          del(a[j], a[j + p]);
 }
 void fwtxor(int a[], int m, int opt) { //(1,1/2) 没有模数的时候用double, 有模数的时候用逆元
  for (int len = 2; len <= m; len <<= 1)
    for (int p = len >> 1, i = 0; i < m; i += len)
      for (int j = i; j < i + p; j++) {
        add(a[j], a[j + p]);
        a[j + p] = (a[j] - 2ll * a[j + p] % mod + mod) % mod;
        a[j] = 1ll * a[j] * opt % mod;
        a[j + p] = 1ll * a[j + p] * opt % mod;
      }
 }
 int a[1 << 17], b[1 << 17], c[1 << 17];
 void mul(int a[], int b[], int c[], int m) {
  for (int i = 0; i < m; i++) c[i] = 1ll * a[i] * b[i] % mod;
 }
 void print(int a[], int m) {
  for (int i = 0; i < m; i++) cout << a[i] << " \n"[i == m - 1];
 }
 int main() {
  int n; cin >> n;
  int m = 1 << n;
  for (int i = 0; i < m; i++) cin >> a[i];
  for (int i = 0; i < m; i++) cin >> b[i];

  fwtor(a, m, 1), fwtor(b, m, 1), mul(a, b, c, m);
  fwtor(a, m, -1), fwtor(b, m, -1), fwtor(c, m, -1), print(c, m);

  fwtand(a, m, 1), fwtand(b, m, 1), mul(a, b, c, m);
  fwtand(a, m, -1), fwtand(b, m, -1), fwtand(c, m, -1), print(c, m);
```

```
  fwtxor(a, m, 1), fwtxor(b, m, 1), mul(a, b, c, m);
  fwtxor(c, m, (mod + 1) / 2), print(c, m);
}
```

生成函数

展开与系数（给定 N）

$$G(x) = (1-x)^{-N} = \sum_{k=0}^{\infty} \binom{N+k-1}{k} x^k$$

系数：$[x^k]\, G(x) = \binom{N+k-1}{k}$

递推：$a_0 = 1, \quad a_{k+1} = a_k \cdot \frac{N+k}{k+1}$

博弈轮

SG 函数模板 (Sprague–Grundy Theorem)

1. 定义

- 对于一个无向图游戏（无环有限博弈）：
  状态 x 的 SG 值定义为：

$$SG(x) = \text{mex}\{SG(y) \mid y \text{ 是 } x \text{ 的后继状态}\} \tag{31}$$

- 其中 mex 表示最小的不在集合中的非负整数。
- 定理：一个局面的 SG 值就是它等价的 Nim 堆大小。
  - 若所有子局面的 SG 值异或和 = 0，则当前局面是 **必败态 (P-position)**。
  - 否则是 **必胜态 (N-position)**。

---

2. mex 函数实现

```
int mex(const vector<int>& s) {
    static vector<int> vis;
    if ((int)vis.size() < (int)s.size() + 5) vis.resize(s.size() + 5);
    for (int x : s) if (x < (int)vis.size()) vis[x] = 1;
    int g = 0;
    while (g < (int)vis.size() && vis[g]) g++;
    for (int x : s) if (x < (int)vis.size()) vis[x] = 0; // reset
    return g;
}
```

Bash Game

每人取`[1, x]`个，那么

- **P-态 (必败态)**

$$n \equiv 0 \pmod{x+1} \tag{32}$$

- **N-态 (必胜态)**

$$n \not\equiv 0 \pmod{x+1} \tag{33}$$

杂项

三分法 (Ternary Search)

实数域三分法

```
// f: 定义在实数上的函数 (double -> double)
// 在区间 [l, r] 上寻找最大值 / 最小值
double ternary_search_real(double l, double r) {
    for (int iter = 0; iter < 200; iter++) { // 200次迭代保证精度1e-18
        double m1 = l + (r - l) / 3.0;
        double m2 = r - (r - l) / 3.0;
        if (f(m1) < f(m2))
            l = m1;  // 如果要求最大值，反过来写
        else
            r = m2;
    }
    return (l + r) / 2.0; // 极值点近似位置
}
```

整数域三分法

```
// f: 定义在整数上的函数 (long long -> long long)
// 在区间 [l, r] 上寻找最大值 / 最小值
#define ll long long
ll ternary_search_int(ll l, ll r) {
    while (r - l > 3) {
        ll m1 = l + (r - l) / 3;
        ll m2 = r - (r - l) / 3;
        if (f(m1) < f(m2))
            l = m1;  // 如果要求最大值，反过来写
        else
            r = m2;
    }
    ll ans = l;
    for (ll i = l; i <= r; i++) {
        if (f(i) < f(ans)) ans = i; // 改成 > 找最大值
    }
    return ans; // 极值点位置
}
```

曼哈顿距离 ↔ 切比雪夫距离的坐标变换

曼哈顿距离 (Manhattan Distance)

$$d_{\text{Manhattan}}(P,Q) = |x_1 - x_2| + |y_1 - y_2| \tag{34}$$

切比雪夫距离 (Chebyshev Distance)

$$d_{\text{Chebyshev}}(P,Q) = \max(|x_1 - x_2|, |y_1 - y_2|) \tag{35}$$

等距曲线是正方形。

曼哈顿 → 切比雪夫

$$(u, v) = \left(\frac{x+y}{2}, \frac{x-y}{2}\right) \tag{36}$$

那么有：

$$d_{\text{Manhattan}}(P, Q) = 2 \cdot d_{\text{Chebyshev}}((u_1, v_1), (u_2, v_2)) \tag{37}$$

切比雪夫 → 曼哈顿

$$(u, v) = (x+y,\ x-y) \tag{38}$$

那么有：

$$d_{\text{Chebyshev}}(P, Q) = \frac{1}{2} \cdot d_{\text{Manhattan}}((u_1, v_1), (u_2, v_2)) \tag{39}$$

更高精度开根

```
LL sqr(LL x){
    LL y = sqrt(x);
    while((y + 1) * (y + 1) <= x) y++;
    while(y * y > x) y--;
    return y;
}
```

内存计算

256MB 6e7的int

long long 3e7

常见函数上下界

$p[20] = 2.4^{18}$

常见公式

```
    1^2 + 2^2 + 3^2 + ..... + x^2 =  x*(x + 1)(2*x+1)/6
```

min(a, b) = a + b - max(a, b)

min-plus , 置换复合满足结合律

斐波那契数列在算法竞赛中的常用性质

基本性质

1. **定义递推**

$$F_0 = 0,\ F_1 = 1, F_n = F_{n-1} + F_{n-2} \tag{40}$$

2. **通项公式（Binet 公式）**

$$F_n = \frac{\varphi^n - \hat{\varphi}^n}{\sqrt{5}}, \quad \varphi = \frac{1+\sqrt{5}}{2},\ \hat{\varphi} = \frac{1-\sqrt{5}}{2} \tag{41}$$

3. **矩阵形式**

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \tag{42}$$

→ 可用 **快速幂** 在 $O(\log n)$ 内求 fib。

模运算与周期性

1. **Pisano 周期**
   任意模 `m` 下，fib 序列 **周期性**。
   应用：当 `n`很大时，先取模周期再计算。
2. **模素数性质**
   对质数 p：

$$F_{p-(5/p)} \equiv 0 \pmod{p} \tag{43}$$

   其中 $(5/p)$ 是勒让德符号。

数学恒等式

1. **和式公式**

$$F_0 + F_1 + \cdots + F_n = F_{n+2} - 1 \tag{44}$$

2. **平方和**

$$F_0^2 + F_1^2 + \cdots + F_n^2 = F_n \cdot F_{n+1} \tag{45}$$

3. **Cassini 恒等式**

$$F_{n+1}F_{n-1} - F_n^2 = (-1)^n \tag{46}$$

4. **倍数与 gcd**
   - 若 $d \mid n$，则 $F_d \mid F_n$。
   - $\gcd(F_m, F_n) = F_{\gcd(m,n)}$。

组合与数论应用

1. **组合解释**
   - 铺砖问题、爬楼梯问题计数 = fib。
   - 常出现在计数 DP 化简。
2. **线性递推通用解法**
   fib 是最经典的二阶递推，可推广到 k 阶递推。
3. **快速倍增算法（Fast Doubling）**

$$F_{2k} = F_k \cdot (2F_{k+1} - F_k), \quad F_{2k+1} = F_{k+1}^2 + F_k^2 \tag{47}$$

   → 也可在 $O(\log n)$ 内求 fib。

竞赛常见考点

- **快速求大 $F_n \bmod m$**：矩阵快速幂 / fast doubling。
- **利用 gcd 性质解题**：下标含 gcd/lcm 时常转化为 fib 性质。

- **Pisano 周期**：fib mod m 的周期性考点。
- **质数与同余**：fib 与素数相关的整除性质。
- **DP 化简**：计数问题最后化简成 fib。
- **矩阵优化**：更复杂递推转移可用矩阵快速幂。